

Programação Web

Laboratórios de Sistemas e Serviços

Mário Antunes

Universidade de Aveiro

2026

JavaScript

O JavaScript (JS) é frequentemente mal compreendido como um “brinquedo de scripting”, mas é uma linguagem sofisticada de alto nível.

1. Tipagem Dinâmica e Tipagem Fraca

- As variáveis não estão ligadas a um *tipo de dados* específico.
- *Porque é que isto importa:* Pode atribuir um Number a uma variável e, mais tarde, atribuir uma String à mesma variável. Isto oferece flexibilidade, mas aumenta o risco de erros em tempo de execução (ex: tentar multiplicar uma string).

```
let x = 42;  
x = "hello";  
console.log(x)
```

2. Orientação a Objetos baseada em Protótipos

- *Como funciona:* Ao contrário das linguagens baseadas em Classes (Java/C++) onde os objetos são instanciados a partir de “moldes” (classes), os objetos JS herdam diretamente de outros objetos (protótipos).
- *Implicação:* A eficiência de memória envolve a clonagem de estruturas existentes em vez de definir hierarquias rígidas.

JavaScript: Visão Geral Detalhada iii

```
let person = {  
  eats: true,  
  hasLegs: 2,  
  walks(){ console.log('I can walk')}  
}  
// definir outro objeto  
let man = {  
  hasBreast: false,  
  hasBeard : true,  
}
```

JavaScript: Visão Geral Detalhada iv

```
// definir o protótipo de man para o objeto person
man.__proto__ = person;
// definir um terceiro objeto
let samuel = {
  age: 23
}
// definir o protótipo de samuel para man
samuel.__proto__ = man;
// acessar ao método walk a partir de samuel
console.log(samuel.walks())
// acessar a hasBeard a partir de samuel
console.log(samuel.hasBeard)
```

3. Execução Single-Threaded

- *A Restrição:* O JS tem uma única **Pilha de Chamadas (Call Stack)**. Só consegue fazer *uma coisa de cada vez*.
- *O Risco:* Se executar um loop matemático pesado (ex: calcular o Pi até mil milhões de dígitos), o separador do browser congela (bloqueio da UI) porque a thread está ocupada.

Paradigmas de Programação

Programação Sequencial (Procedimental)

Este é o modelo utilizado em C básico, Fortran ou scripts simples de Python.

A Lógica:

1. O programa inicia.
2. A Linha 1 é executada.
3. A Linha 2 é executada.
4. **A Linha 3 pede um input (`scanf`, `input()`).**
5. O programa **PARA** (bloqueia) e espera pelo utilizador.
Nada mais acontece até o utilizador carregar em Enter.

Porque é que isto falha na UI: Numa interface web, não podemos “**parar**” o motor de renderização para esperar por um clique do rato. Se o fizéssemos, os botões não teriam animação e os gifs não correriam.

Programação Baseada em Eventos i

As interfaces modernas (Web, Windows, macOS) utilizam uma arquitetura **Baseada em Eventos**.

A Lógica:

1. O programa inicia (Inicialização).
2. Define “Handlers” (funções à espera de gatilhos específicos).
3. Entra no **Event Loop** (Ciclo de Eventos).
4. O programa fica num **Estado de Escuta**.

O “Princípio de Hollywood”:

- *Não nos chame, nós chamamos-te.*
- O código não pergunta “O utilizador clicou?”. Em vez disso, o browser interrompe o código dizendo “Acabou de acontecer um clique, executa a Função de Clique.”

```
{ width=65% }
```

Como é que uma ação física se torna execução de código?

1. **Nível de Hardware:** O utilizador move o rato. O hardware do rato envia um sinal elétrico (interrupção) ao CPU.
2. **Nível de SO:** O Sistema Operativo (Windows/Linux) interpreta este sinal como uma mudança de coordenadas e desenha o cursor a mover-se.
3. **Nível de Browser:** A janela do browser vê que o cursor está sobre um botão HTML específico e que o botão do rato foi premido.

4. **O Evento:** O browser cria um Objeto Event de JavaScript contendo detalhes (coordenadas X/Y, qual o botão, carimbos temporais).
5. **O Listener:** O browser verifica: *Este elemento HTML tem um listener associado?*
6. **Execução:** Se sim, a função JS registada é enviada para a pilha de execução.

JavaScript na Página Web

O Document Object Model (DOM)

O Conceito: Quando escreve um ficheiro HTML, este é apenas uma string de texto. O browser analisa (parse) esta string para uma estrutura em memória chamada DOM.

- **HTML:** `<div id="app"></div>` (Texto no disco rígido)
- **DOM:** `HTMLDivElement` (Objeto na RAM)

Porque é que o JS usa o DOM: O JavaScript não pode editar o ficheiro de texto no servidor. Edita o **Objeto na RAM**. O motor de renderização do browser vigia constantemente o DOM; quando o JS atualiza o objeto DOM, o browser redesenha o ecrã.

O HTML é analisado sequencialmente (de cima para baixo). Quando o analisador vê uma tag `<script>`, pausa a análise do HTML para descarregar e executar o script. Isto cria problemas:

1. O “Truque do Fundo do Body”

- *Técnica:* Colocar o `<script>` logo antes do `</body>`.
- *Raciocínio:* Garante que todos os elementos HTML existem no DOM antes de o script tentar encontrá-los.

2. O Atributo defer (Padrão Moderno)

```
<script src="app.js" defer></script>
```

- *Comportamento:* O script é descarregado em segundo plano (paralelo) enquanto o HTML é analisado.
- *Execução:* O browser garante que o script só será executado **após** o HTML estar totalmente analisado, mas **antes** do evento DOMContentLoaded.
- *Benefício:* Tempos de carregamento de página mais rápidos e acesso seguro ao DOM.

O Event Loop (Detalhe Técnico)

Como é que o JS single-threaded gere tarefas assíncronas (como procurar dados) sem congelar?

1. **Pilha de Chamadas (Call Stack):** Executa código síncrono (LIFO).
2. **Web APIs:** Quando chama `setTimeout` ou `fetch`, o “trabalho” é delegado às threads C++ do Browser (não à thread do JS).
3. **Fila de Callbacks (Callback Queue):** Quando a Web API termina, coloca a sua função de callback numa Fila.
4. **O Ciclo (Loop):** O Event Loop verifica: “A Pilha está vazia?”
 - Se **NÃO**: Espera.
 - Se **SIM**: Move o primeiro item da Fila para a Pilha.

É por isto que o `setTimeout(fn, 0)` não corre imediatamente — espera que a pilha fique limpa.

Exemplos de JS

1. Gestão de Eventos de Rato

Usamos `addEventListener`. Esta é a fase de registo da programação baseada em eventos.

```
const box = document.querySelector('#box');  
// O objeto 'event' é passado automaticamente pelo browser  
function handleMove(event) {  
  // Atualiza o texto com as coordenadas do rato  
  box.textContent = `X: ${event.clientX}, Y: ${event.clientY}`;  
  // Estilização dinâmica baseada em lógica  
  if (event.clientX > 500) {  
    box.style.backgroundColor = 'red';  
  } else {  
    box.style.backgroundColor = 'blue';  
  }  
}  
// Subscrever o evento 'mousemove'  
box.addEventListener('mousemove', handleMove);
```

2. Conteúdo Dinâmico (Biblioteca de Fotos)

Podemos criar a interface programaticamente. É assim que o React/Vue funcionam internamente (abordagem Imperativa).

```
const urls = ['img1.jpg', 'img2.jpg'];
const container = document.getElementById('gallery');

urls.forEach(url => {
  // 1. Criar Elemento: Cria um objeto órfão em memória
  const img = document.createElement('img');
  // 2. Configurar Objeto: Definir propriedades
  img.src = url;
  img.className = 'thumbnail';
  // 3. Associar Evento: Torná-lo interativo imediatamente
  img.addEventListener('click', () => {
    console.log("Clicou em " + url);
  });
  // 4. Montar: Inserir na árvore DOM ativa.
  container.appendChild(img);
});
```

3. Dados Assíncronos (Fetch API)

Procurar dados de uma API demora tempo (latência).
Usamos **Promises** (`async/await`) para evitar o bloqueio.

```
async function getData() {  
  try {  
    // 'await' liberta a thread até que a Promise seja resolvida.  
    // A UI permanece responsiva durante esta pausa.  
    const response = await fetch('https://api.data.gov/users');  
    // A análise de JSON também é assíncrona (gere streams)  
    const data = await response.json();  
    console.log(data); // Corre apenas após o fim da rede  
  } catch (error) {  
    // Gere falhas de rede (404, 500, Offline)  
    console.error("Falha no Fetch:", error);  
  }  
}
```

4. Comunicação em Tempo Real (WebSockets)

HTTP vs. WebSockets:

- **HTTP:** O Cliente pergunta, o Servidor responde, a ligação fecha. (Stateless).
- **WebSocket:** O Cliente realiza um “Handshake”, a ligação é atualizada para um socket TCP, a ligação permanece aberta.

```
const socket = new WebSocket('ws://localhost:8080');  
// Evento: Ligação Estabelecida  
socket.onopen = () => {  
    console.log("Ligado ao Servidor de Chat");  
    socket.send("Utilizador entrou");  
};  
// Evento: O servidor enviou-nos dados  
socket.onmessage = (event) => {  
    // Isto dispara sempre que o servidor envia dados. Sem polling!  
    const message = JSON.parse(event.data);  
    displayMessage(message);  
};
```

Depuração (Debugging) no Browser

O Desafio das Linguagens Interpretadas i

Ao contrário de C, C++ ou Rust, o JavaScript é uma linguagem **Interpretada** (ou compilada por JIT).

Linguagens Compiladas (C/C++):

- O compilador analisa todo o código **antes** da execução.
- Erros de sintaxe e incompatibilidades de tipos são detetados em **Tempo de Compilação**.
- *Resultado*: Não pode executar o programa até que estes erros sejam corrigidos.

Linguagens Interpretadas (JavaScript):

- O browser lê e executa o código linha a linha (ou bloco a bloco) em **Tempo de Execução (Runtime)**.
- *Resultado*: A aplicação pode carregar perfeitamente e correr durante minutos.
- **O Crash**: O erro só ocorre quando o fluxo de execução atinge a linha específica com o erro (ex: quando um utilizador clica num botão específico).

Consequência: O “funciona na minha máquina” é comum. Pode não encontrar o erro porque não ativou o caminho de execução específico que contém o bug.

Depurar aplicações Web introduz uma desconexão entre onde se **escreve** código e onde se **executa** código.

1. A Troca de Contexto:

- Escreve código num **IDE** (VS Code), que tem análise estática e linting.
- Executa código no **Browser** (Chrome/Firefox).
- Quando ocorre um erro, ele aparece na Consola do Browser, não imediatamente no seu editor de texto.

2. O Problema da “Caixa Preta”:

- O browser corre frequentemente código “minificado” ou “empacotado” (para poupar largura de banda).
- Um erro na linha 1 de `bundle.js` é inútil para o desenvolvedor.
- *Solução*: Confiamos nos **Source Maps**, que dizem ao browser como mapear o código em execução de volta aos seus ficheiros originais.

1. Depuração por “Printf” (`console.log`)

- O método mais antigo. Imprime variáveis na consola do browser para inspecionar o estado.
- *Prós*: Rápido, simples.
- *Contras*: Desarruma o código, exige limpeza, não pausa a execução.

2. A Palavra-chave `debugger` ;

- Colocar a instrução `debugger` ; no seu código força o browser a **pausar a execução** (breakpoint) nessa linha.
- Pode então percorrer o código linha a linha.

3. DevTools do Browser (O separador Sources)

- Os browsers modernos (Chrome/Firefox) têm depuradores integrados que rivalizam com IDEs de desktop.
- Pode definir breakpoints, vigiar variáveis e inspecionar a Pilha de Chamadas diretamente no browser.

Frameworks Frontend Modernos

O Problema de “Estado vs. Vista”

Em aplicações complexas (ex: Facebook, Spotify), manter a UI (Vista) sincronizada com os dados (Estado) usando Vanilla JS é propenso a erros.

As Frameworks resolvem isto através de:

1. **Programação Declarativa:** Define-se *o que* a UI deve parecer para um determinado estado, não *como* a atualizar.
2. **Componentização:** Divisão da UI em pedaços reutilizáveis e isolados.

Desenvolvido pelo Facebook (Meta). O React é tecnicamente uma **Biblioteca**, não uma Framework, focado exclusivamente na camada da Vista.

Conceitos-Chave:

1. **DOM Virtual:** O React mantém uma cópia leve do DOM em memória. Quando o estado muda, calcula a “diferença” (diff) e atualiza apenas as partes alteradas do DOM real.
2. **JSX (JavaScript XML):** Extensão de sintaxe que permite escrever HTML dentro de JS.
3. **Fluxo de Dados Unidirecional:** Os dados fluem para baixo (Pai -> Filho).

Exemplo de React

Note a natureza **Declarativa**. Não chamamos `appendChild`.
Devolvemos a estrutura que queremos.

```
import React, { useState } from 'react';
function ImageGallery() {
  // Hook de Estado: Quando 'images' muda, a UI atualiza-se auto.
  const [images, setImages] = useState([
    { id: 1, url: 'img1.jpg' }
  ]);
  return (
    <div id="gallery">
      /* Loop dentro do JSX */
      {images.map(img => (
        <img key={img.id} src={img.url}
          className="thumbnail" />
      ))}
    </div>
  );
}
```

Angular: A Framework

Desenvolvido pela Google. O Angular é uma **Framework** completa. Inclui roteamento, clientes HTTP e gestão de formulários de raiz.

Conceitos-Chave:

1. **TypeScript:** Obrigatório. Adiciona tipagem estática (Interfaces, Classes) ao JS para segurança.
2. **Injeção de Dependências (DI):** Sistema integrado para gerir serviços e estado.
3. **Ligação de Dados Bidirecional (Two-Way Data Binding):** Alterações na UI atualizam o Estado; Alterações no Estado atualizam a UI (automaticamente).
4. **DOM Real:** O Angular opera diretamente no DOM, mas utiliza um mecanismo sofisticado de Detecção de Alterações (Zones).

Exemplo de Angular i

O Angular separa a Lógica (Typescript) da Vista (Template HTML).

Lógica do Componente (gallery.component.ts)

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-gallery',
  templateUrl: './gallery.component.html'
})
export class GalleryComponent {
  // Array Tipado
  images: Array<{url: string}> = [{ url: 'img1.jpg' }];
}
```

Template (gallery.component.html)

```
<div id="gallery">  
  <img *ngFor="let img of images"  
    [src]="img.url"  
    class="thumbnail">  
</div>
```

Comparação Resumida

Característica	Vanilla JS	React	Angular
Paradigma	Imperativo	Declarativo	Declarativo
Linguagem	JavaScript	JS + JSX	TypeScript
DOM	Acesso Direto	DOM Virtual	DOM Real + Zones
Escala	Scripts pequenos	Apps Médias/Grandes	Apps Empresariais
Curva de Aprendizagem	Baixa	Média	Alta

Backends

O **Node.js** não é uma linguagem; é um **Ambiente de Execução**. Pega no Motor V8 do Chrome e adiciona ligações em C++ para o Sistema de Ficheiros (FS) e Redes, permitindo que o JS corra em servidores.

NPM (Node Package Manager):

- Gere dependências (bibliotecas).
- **package.json**: O manifesto do projeto. Lista quais as bibliotecas necessárias (dependencies) e como executar o projeto (scripts).

Servidor Express Simples

O Express é a framework padrão para Node. Simplifica o roteamento.

```
// Importar biblioteca express
const express = require('express');
const cors = require('cors'); // Middleware para Segurança
const app = express();
// Ativar CORS: Permite que o nosso JS do browser (de uma origem diferente)
// procure dados neste servidor. Sem isto, o browser bloqueia.
app.use(cors());
// Definir uma Rota (Endpoint)
app.get('/api/hello', (req, res) => {
  // Enviar resposta JSON
  res.json({
    msg: "Olá Mundo",
    serverTime: Date.now()
  });
});
app.listen(3000, () => console.log("A correr na porta 3000"));
```

Embora o Node.js partilhe a linguagem com o frontend, o **Python** é dominante em Ciência de Dados e IA.

Características do FastAPI:

1. **Assíncrono:** Usa o `async def` do Python (padrão ASGI), tornando-o muito mais rápido que o Flask/Django.
2. **Dicas de Tipo (Type Hints):** Valida dados automaticamente.
3. **Swagger UI:** Gera automaticamente um site de documentação (`/docs`) para a sua API.

Exemplo de FastAPI

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# Configuração de CORS
# Permitindo explicitamente o container/origem do frontend
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Em prod, substituir * pelo domínio específico
    allow_methods=["*"],
)

@app.get("/api/items")
async def read_items():
    # O Dicionário Python é convertido automaticamente para JSON
    return [
        {"name": "Item 1", "price": 10.5},
        {"name": "Item 2", "price": 20.0}
    ]
```

Temos duas aplicações separadas:

1. **Frontend:** HTML/JS estático servido pelo Nginx (ou uma app React/Angular compilada).
2. **Backend:** API Python/Node que processa dados.

Precisamos de as executar juntas e garantir que comunicam.

Configuração do Docker Compose i

O `docker-compose.yml` orquestra aplicações multi-container.

```
services:
  backend-api:
    build: ./backend_folder           # Criar imagem a partir do Dockerfile
    container_name: py_api
    ports:
      - "8000:8000"                 # Expor porta 8000 para o host
    volumes:
      - ./backend_folder:/app        # Atualização em tempo real das mud

  frontend-web:
    image: nginx:alpine              # Usar Nginx pré-construído
    container_name: my_website
    ports:
      - "8080:80"                   # O browser acede a localhost:8080
    volumes:
      - ./frontend_folder:/usr/share/nginx/html # Injetar o nosso HTML
    depends_on:
      - backend-api                  # Esperar que a API inicie
```

Conceito Crítico de Redes:

- **Browser para o Backend:** Quando o seu JavaScript corre no *browser*, está a correr na *Máquina do Utilizador*. Portanto, o URL do fetch no JS deve apontar para `http://localhost:8000` (a porta exposta pelo Docker para a máquina host), não para o nome interno do container.

JavaScript e a Web

- [MDN Web Docs \(Mozilla\)](#) - A bíblia do desenvolvimento web.
- [JavaScript.info](#) - Mergulho profundo na linguagem moderna.
- [What the heck is the event loop anyway?](#) (Philip Roberts)
- Visualização essencial do runtime do JS.

Frameworks

- [React Documentation](#) - Documentação oficial (recentemente reescrita).
- [Angular University](#) - Tutoriais abrangentes para Angular.

Backend & DevOps

- [Node.js Best Practices](#) - Padrões de arquitetura.
- [FastAPI User Guide](#) - Excelente documentação com exemplos interativos.